

LSTM Auto Encoders Project

Amit Zulan, Dvir Ben Zikri

January 2025

1 Toy Example with Synthetic Data

1.1 Synthetic Data

In this section we will train the LSTM Auto-Encoder, with a toy synthetic data that was generated in following manner:

We generated 10,000 sequences of length 50, with a single feature, each within 0 and 1.

Afterwards, an index in the range $[20, 30]$ is chosen, and all the values within 5 indices of the chosen index are multiplied by 0.1.

Here are the plots of 3 of the signals generated:

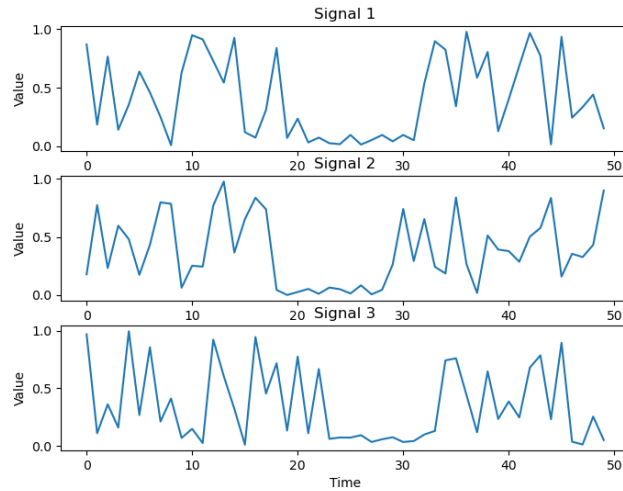


Figure 1: Synthetic Generated Signals

1.2 Training of the Auto Encoder

The model was trained with the following hyper-parameters:

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	1e-2
Gradient Clipping	0.1	2	0.1
Hidden Size	1	25	25
Epochs	-	-	1000
Batch Size	-	-	64

Table 1: Hyper-Parameters of the Toy Auto Encoder.

The following plots show the original signals, and their reconstructions:

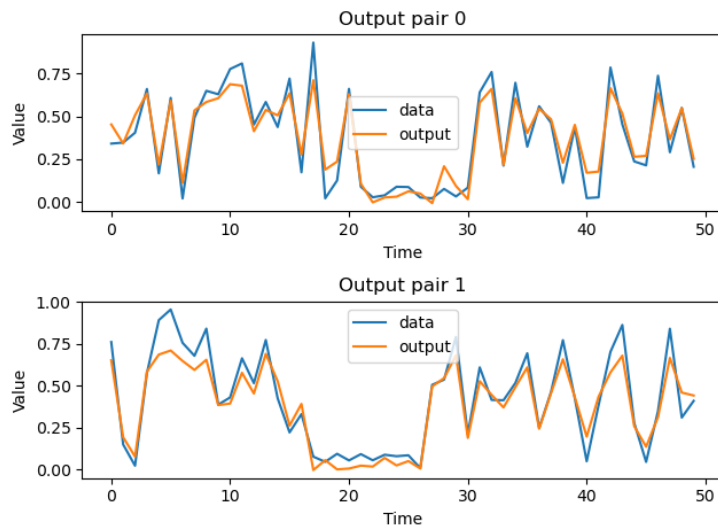


Figure 2: Signals and Their Reconstruction

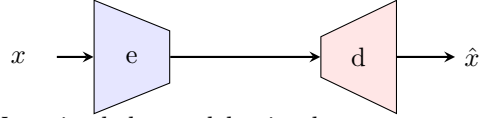
Overall, the Auto Encoder learns this toy dataset pretty well, even though it is practically noise. We would have guessed a-priori that the model should predict all changes to their mean (0.5) and the modified area to (0.05) - that did happen when we let the model trained for only 10 to 50 epochs but after that it is moving towards the perfect fit. Since the hidden state size does not cover all the sequence $\times$ features in total (50×1) and the fact that it is random noise, it can not learn the identity function.

Moreover, we can see that when the model does not perfectly fit, it tries to predict values closer to the nearby average of the sequence (0.5) while still predicting values that are close to the true values.

2 MNIST Reconstruction and classification

2.1 Reconstruction

We trained an LSTM Auto-Encoder with the following architecture.



We trained the model using hyperparameter search using Optuna.
We search the following parameter space:

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	1e-3
Gradient Clipping	0.1	2	1.7
Hidden Size	1	128	128
Epochs	-	-	10
Batch Size	-	-	64

Table 2: Hyper-Parameters of the MNIST Auto Encoder.

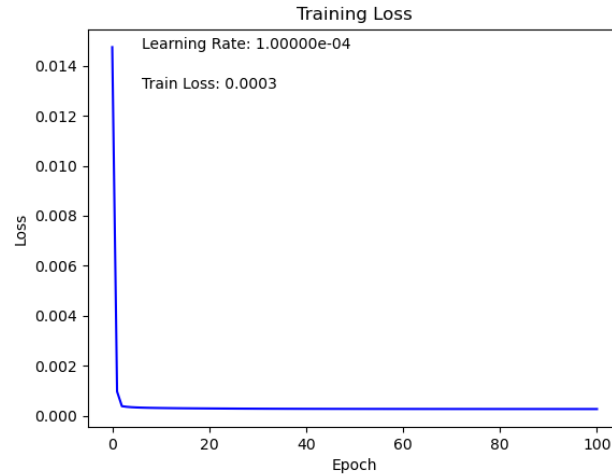


Figure 3: MNIST Auto Encoder Train Loss

In the following, we present the reconstruction of 3 digits:

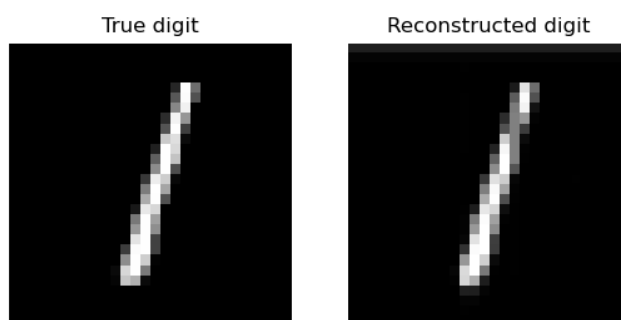


Figure 4: Reconstruction of 1

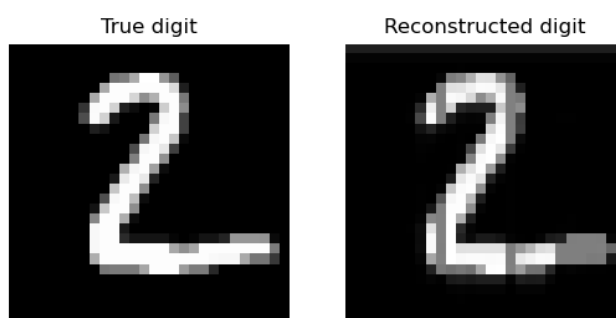


Figure 5: Reconstruction of 2

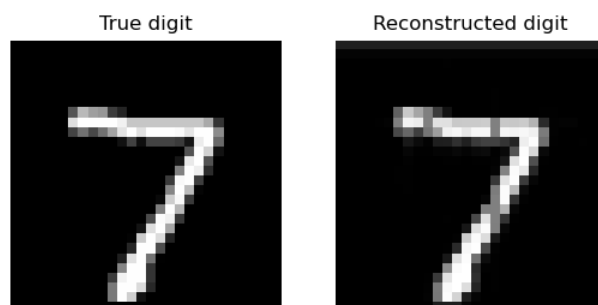
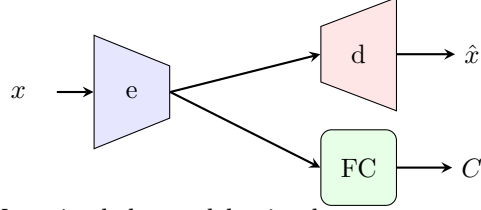


Figure 6: Reconstruction of 7

We can see that the AE is reconstructing the digits pretty well.

2.2 Classification

The classification model was constructed with an LSTM encoder, LSTM decoder, and a fully connected layer.



We trained the model using hyperparameter search using Optuna. We search the following parameter space:

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	1e-3
Gradient Clipping	0.1	2	1.7
Hidden Size	1	128	128
Epochs	-	-	50
Batch Size	-	-	64

Table 3: Hyper-Parameters of the model.

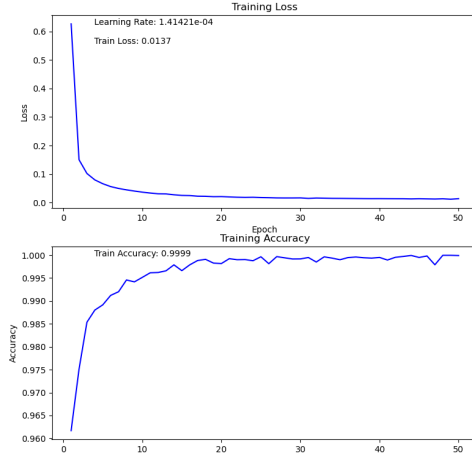


Figure 7: MNIST Classifier train loss and accuracy vs epochs

The above model classified the test set with 98.73% accuracy.

2.3 Pixel Wise

With the same architecture as above, we performed another hyper-parameter search with the following parameters:

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	1e-2
Gradient Clipping	0.1	2	1.2
Hidden Size	1	128	101
Epochs	-	-	10
Batch Size	-	-	64

Table 4: Hyper-Parameters of the model.

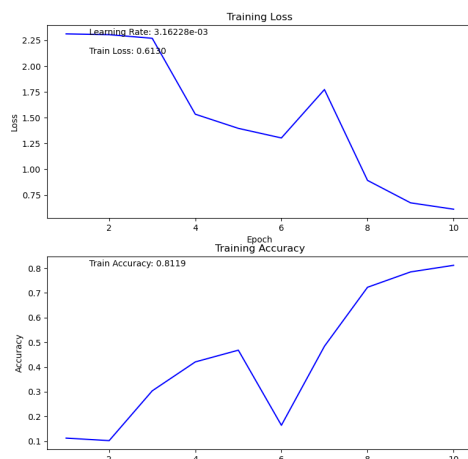


Figure 8: MNIST Classifier train loss and accuracy vs epochs

The above model was trained for 10 epochs with batch size of 64, and classified the test set with 80.93%

The pixel-wise model, performs worse than the row-wise, this makes sense, due to the short memory of LSTMs we talked about in class. Because of exploding and vanishing gradients, more specifically vanishing gradients, the model is having issues, with longer sequences.

3 S&P 500 Reconstruction and Regression

3.1 Example Stocks

Below is the daily maximum value of the AMZN and GOOGL stocks.

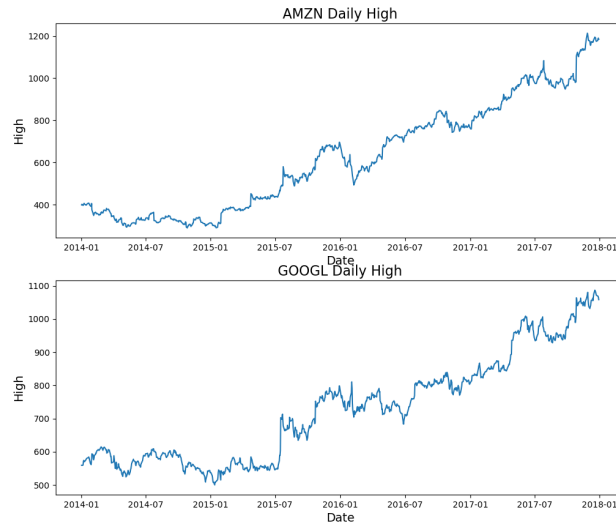


Figure 9: AMZN and GOOGL daily max

3.2 S&P Reconstruction

We search the following parameter space:

The batch size has increased, due to spikes in training loss, in lower batch sizes.

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	0.032
Gradient Clipping	0.1	2	1.3
Hidden Size	1	150	51
Epochs	-	-	100
Batch Size	-	-	512

Table 5: Hyper-Parameters of the model.

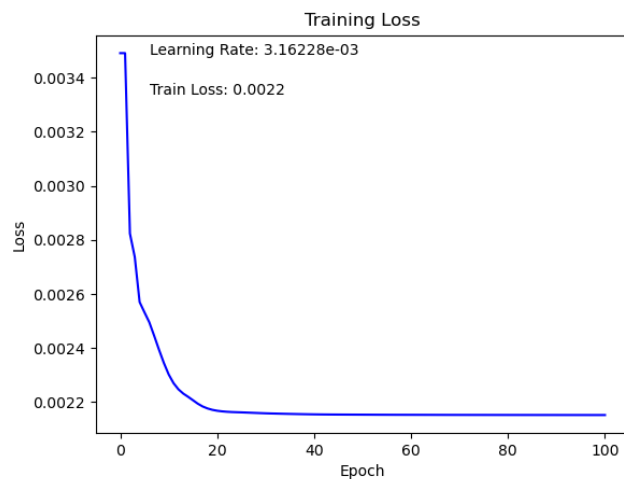


Figure 10: AE train loss vs epochs

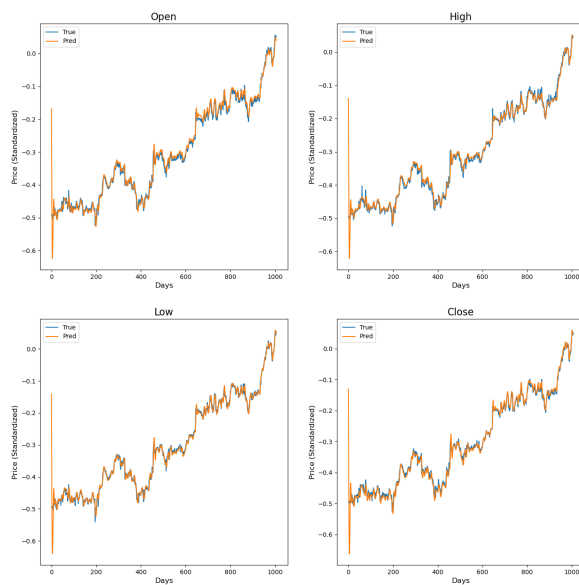


Figure 11: AE reconstruction of first stock

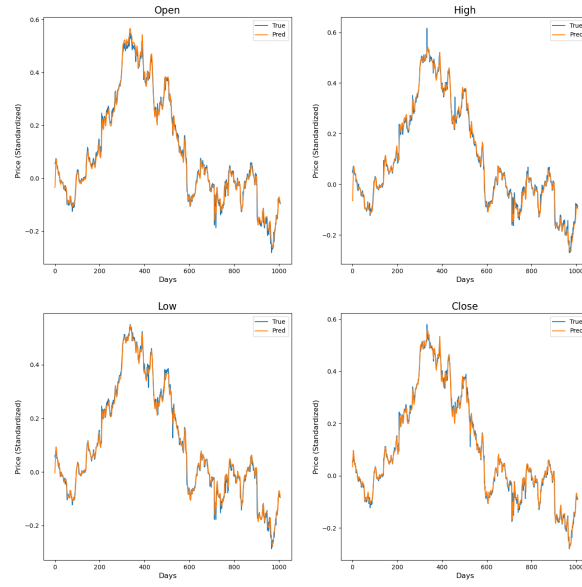


Figure 12: AE reconstruction of second stock

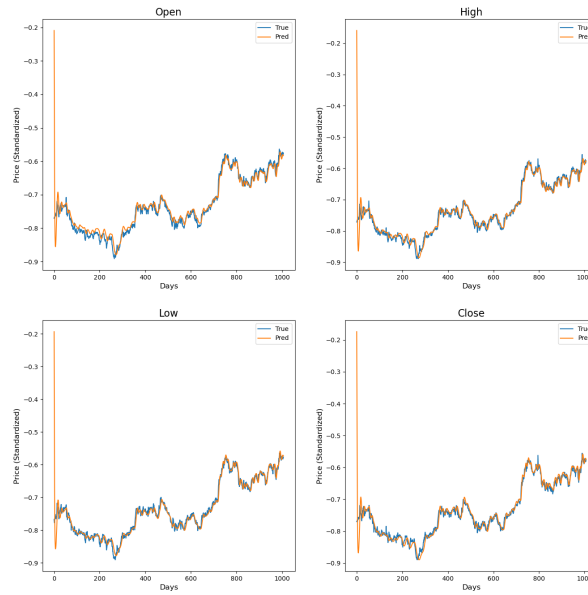


Figure 13: AE reconstruction of third stock

3.3 One Step Prediction

The model was trained with the following hyper-parameters:

Parameter	min	max	Selected Value
Learning Rate	1e-5	1e-1	0.032
Gradient Clipping	0.1	2	0.5
Hidden Size	1	150	63
Epochs	-	-	100
Batch Size	-	-	512

Table 6: Hyper-Parameters of the model.

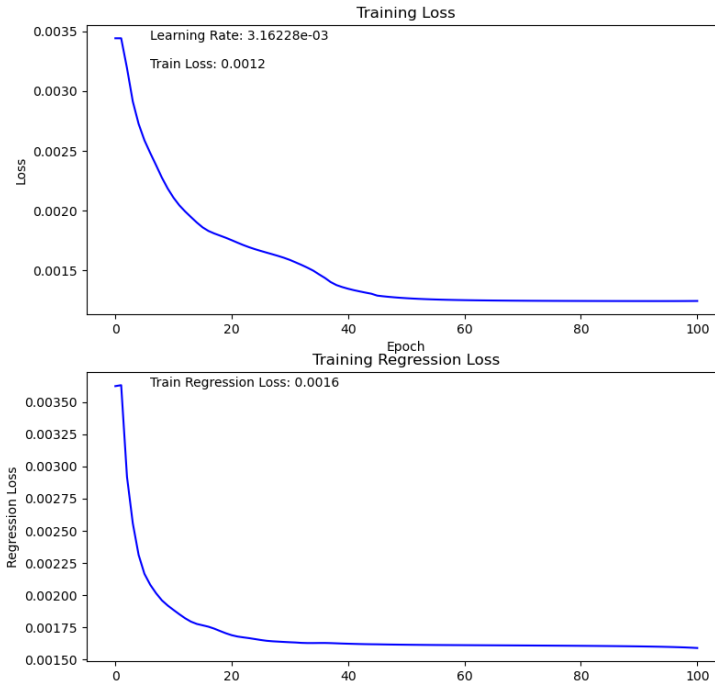


Figure 14: AE train loss vs epochs and AR loss vs epochs (predictions)

Multi step prediction vs One step are show in the following section.

3.4 Multi Step Prediction

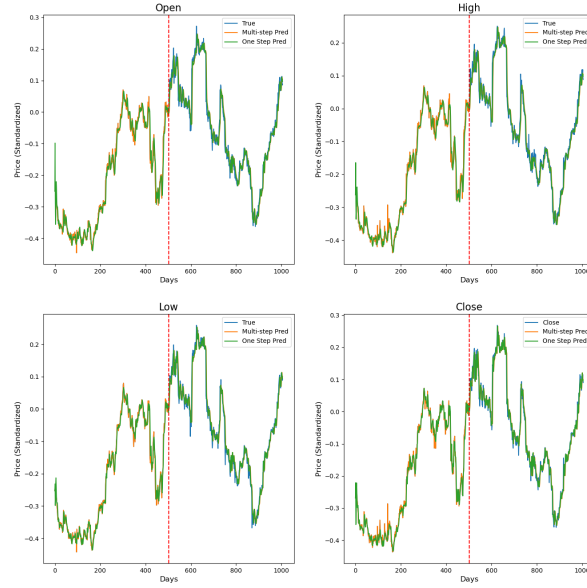


Figure 15: First Stock Prediction, Original vs Multi step vs One step

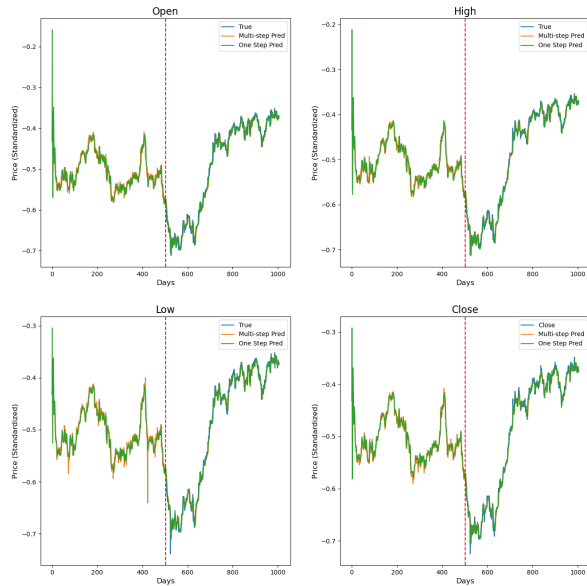


Figure 16: Second Stock Prediction, Original vs Multi step vs One step

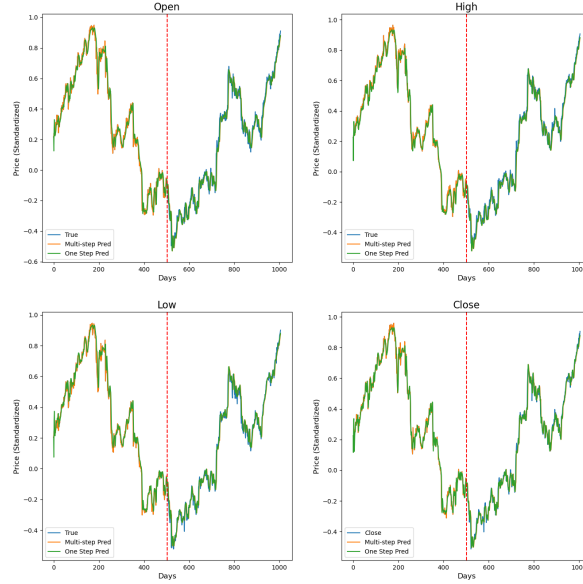


Figure 17: Third Stock Prediction, Original vs Multi step vs One step

Watching closely we can see that the multi-step prediction (generating one output after a true sequence of length N repeating for the whole N future values) and the one step prediction (using the forward of the model, reconstructing $\hat{y}$) are very similar meaning our auto regressive model is quite good at learning and predicting one "future" value.

Another thing we can notice is the fact that both models - AutoEncoder and AutoRegressor (one-step prediction) are not so good at the beginning this can be explained by the fact that they lack information there, and have less "time" data to learn the correct trend the data has, it is more difficult for them to predict or reconstruct that data.